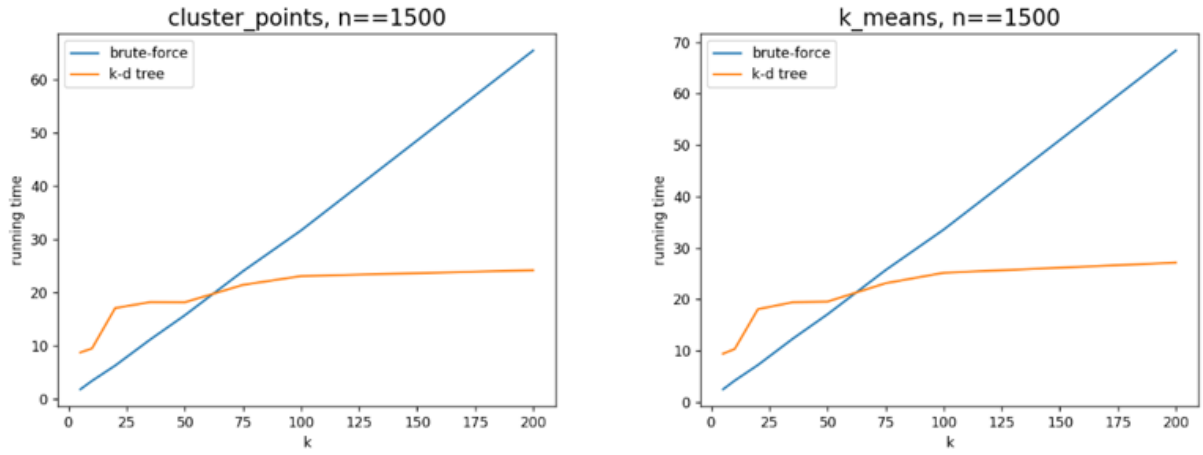


Figure 12.8 Running time comparison between `classifyPoints` (left) and `k_means` (right) implementations using brute-force search and k-d tree's nearest neighbor. The running times are shown as functions of `k`, the number of centroids, for fixed values of `n`, the size of the dataset.
The charts are plotted for `n==1500`, but they show the same trend for all tested values of `n > 1000`.
Comparing the two plots it's also evident how the `classifyPoints` method accounts for most of the running time of this k-means implementation.

If you look closely at the 4 charts in figure 12.7, you can notice how the line for the k-d tree implementation is more stable through the various values of k, while the slope of the blue line for the brute-force search algorithm becomes steeper as k grows.

This trend is even more apparent if we look at the data from a different angle, by keeping n fixed and plotting the running time as a function of k, as we do in figure 12.8: